

Q1:

Worst case happens when the pivot always splits the array into:

- One subarray of size $n - 1$
- One subarray of size 0

For quicksort choosing the first element as pivot, this happens on already sorted arrays.

The recurrence relation is:

$$T(n) = T(n - 1) + cn$$

where cn accounts for the comparisons in the partition step. Expanding the recurrence:

$$\begin{aligned} &= T(n - 1) + cn \\ &= T(n - 2) + c(n - 1) + cn \\ &= T(n - 3) + c(n - 2) + c(n - 1) + cn \\ &\quad \dots \\ &= c(1 + 2 + \dots + n) \end{aligned}$$

The sum of the first n integers is:

$$1 + 2 + \dots + n = \frac{n(n + 1)}{2}$$

Thus:

$$T(n) = c \frac{n(n + 1)}{2} = O(n^2)$$

Worst-case complexity: $O(n^2)$

Q2:

Worst-Case Input

A sorted array of 16 elements produces the worst case when the pivot is always the first element:

[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16]

Manual Quicksort Steps

1. Pivot = 1 $\rightarrow$ left = [], right = [2..16]
 2. Pivot = 2 $\rightarrow$ left = [], right = [3..16]
 3. Pivot = 3 $\rightarrow$ left = [], right = [4..16]
 - ...
 4. Pivot = 15 $\rightarrow$ left = [], right = [16]
- Each recursive step only reduces the right subarray by 1.
 - Total number of comparisons: $1 + 2 + \dots + 15 = 120$.
 - Recursion depth = 16.
 - Array is now fully sorted.

Q4:

We ran quicksort on sorted arrays of increasing size to simulate the worst-case scenario. As expected, the measured running times increased rapidly as the input size grew, showing an approximately quadratic growth pattern. This behavior matches the theoretical worst-case complexity of $O(n^2)$, because in each recursive step the pivot divides the array into one subarray of size $n - 1$ and another of size 0, leading to a recursion depth of n and a total number of comparisons proportional to $n(n - 1)/2$. While the measured times are not exact due to Python overhead and timing variations, they closely follow the expected quadratic trend. This confirms that for sorted arrays and choosing the first element as the pivot, quicksort exhibits its worst-case performance, validating the theoretical analysis derived in Question 1.